

Does globally optimised multi-agent routing via Genetic Algorithms produce better evacuation outcomes than individual greedy path routing?

And under what conditions does the greedy path selection fail?

Hypothesis

Allowing all agents to follow the best path will result in congested nodes resulting in delays as the greedy algorithm can only focus on the path and not the wider population level congestion. The genetic algorithm will inherently account for this collective congestion in its fitness function and is likely a realistic approximation for real life if people where to communicate in evacuation situations (i.e. use different exits rather than the shortest one).

```
In [1]: %load_ext autoreload
%autoreload 2
```

Environment Set Up

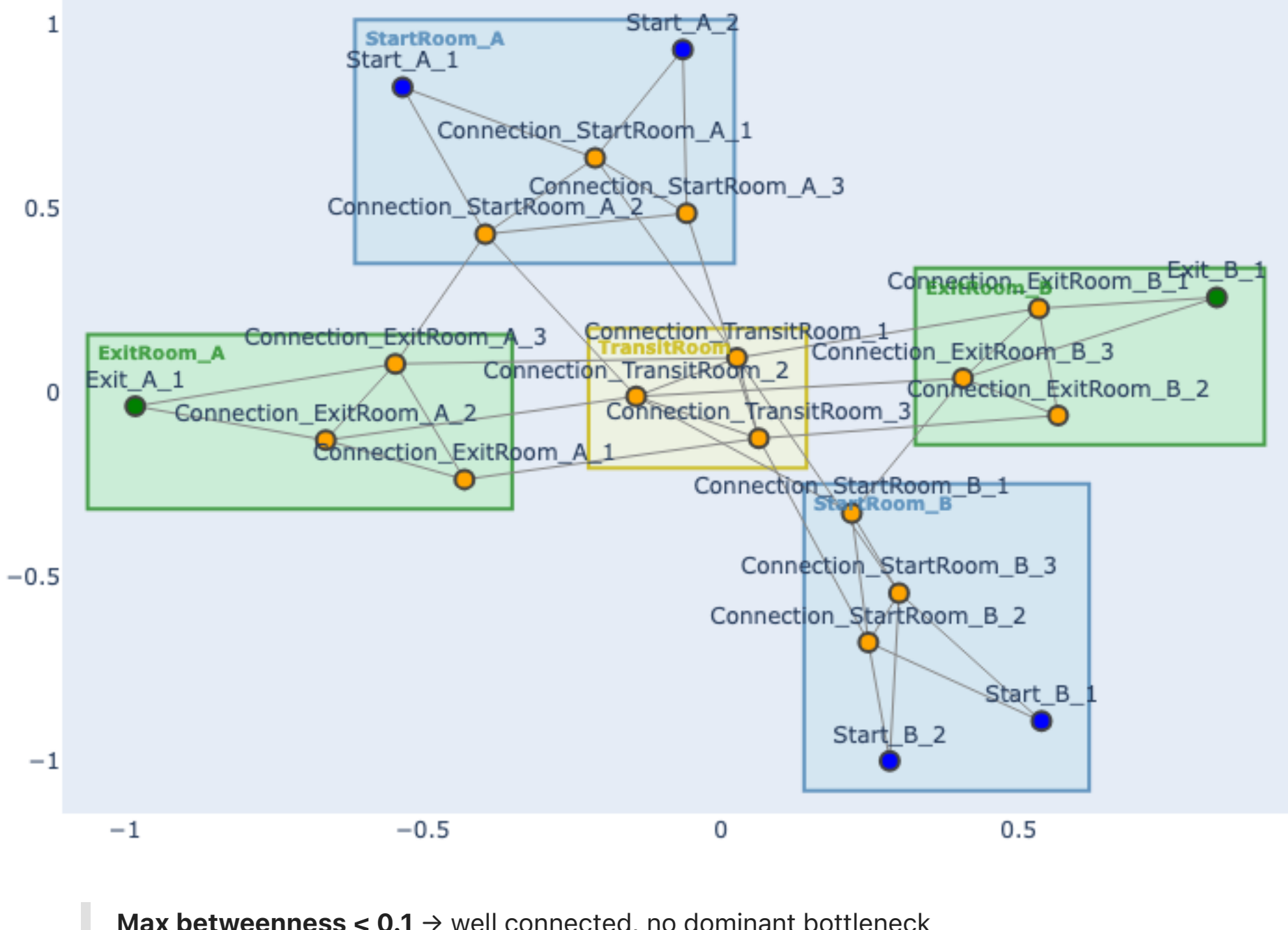
```
In [2]: import numpy as np
from tqdm import tqdm

from utils.environment import Environment
from multi_agent_ga import simulate_ga_evacuation, simulate_greedy_evacuation
from utils.algorithm_evaluation import AlgorithmComparison

In [3]: city_name = "bottleneck_severe_small"
city = Environment(city_name=city_name)
city.summarise_city()
```

Bottleneck severe small has 4 starting points and 2 exit points with a bottleneck score of 0.269

bottleneck_severe_small



Max betweenness < 0.1 → well connected, no dominant bottleneck
Max betweenness 0.1-0.3 → moderate bottleneck, some critical junctions
Max betweenness > 0.3 → severe bottleneck, network heavily dependent on few nodes

In the following experiments the scenarios represent individual agents with the following characteristics:

- Walking speed - some agents walk faster/slower than other agents.
- Each agent has their own starting position (this does not vary between the genetic algorithm and greedy algorithm) # TODO: Check this...
- Each agent is considered an "individual" they have no connection to any other agent - i.e. no agents need to stay with each other like families.

The following cell defines the number of experiments which should be run for comparison and defines the random seeds which will be used across both experiments - this ensures the agents have the same start point in the city.

```
In [4]: n_experiments = 30

rng = np.random.default_rng(42)
algorithm_seeds = rng.integers(0, 10**6, size=n_experiments)

num_agents = 20
simulation_params = {"congestion": True, "walking": False, "fitness": "max"}
```

Example Greedy Algorithm

The below code shows how the greedy algorithm works - producing the final output metrics for this scenario.

```
In [5]: greedy_solution, greedy_evaluation = simulate_greedy_evacuation(
    num_agents=num_agents,
    simulation_params=simulation_params,
    algorithm_seed=algorithm_seeds[0],
    city=city,
    verbose=True
)
```

===== FINAL RESULTS =====
Avg path length: 4.75 | Total time: 39.00 | Average time: 27.05 | Congestion index: 75% | Avg Congestion delay: 22.30 | Exit utilisation: 0.00 | Avg Path Efficiency: 1.00
=====

Genetic Algorithm

The below code shows how the genetic algorithm works - producing metrics showing the performance of the evolutions over time (if you want to see every evolution set `verbose` to 1) as well as the final output metrics for this scenario.

```
In [6]: ga_solution, ga_evaluation = simulate_ga_evacuation(
    city=city,
    num_agents=num_agents,
    population_size=50,
    algorithm_seed=algorithm_seeds[0],
    max_evolution=50,
    simulation_params={"congestion": True, "walking": False, "fitness": "max"},
    verbose=5,
)
```

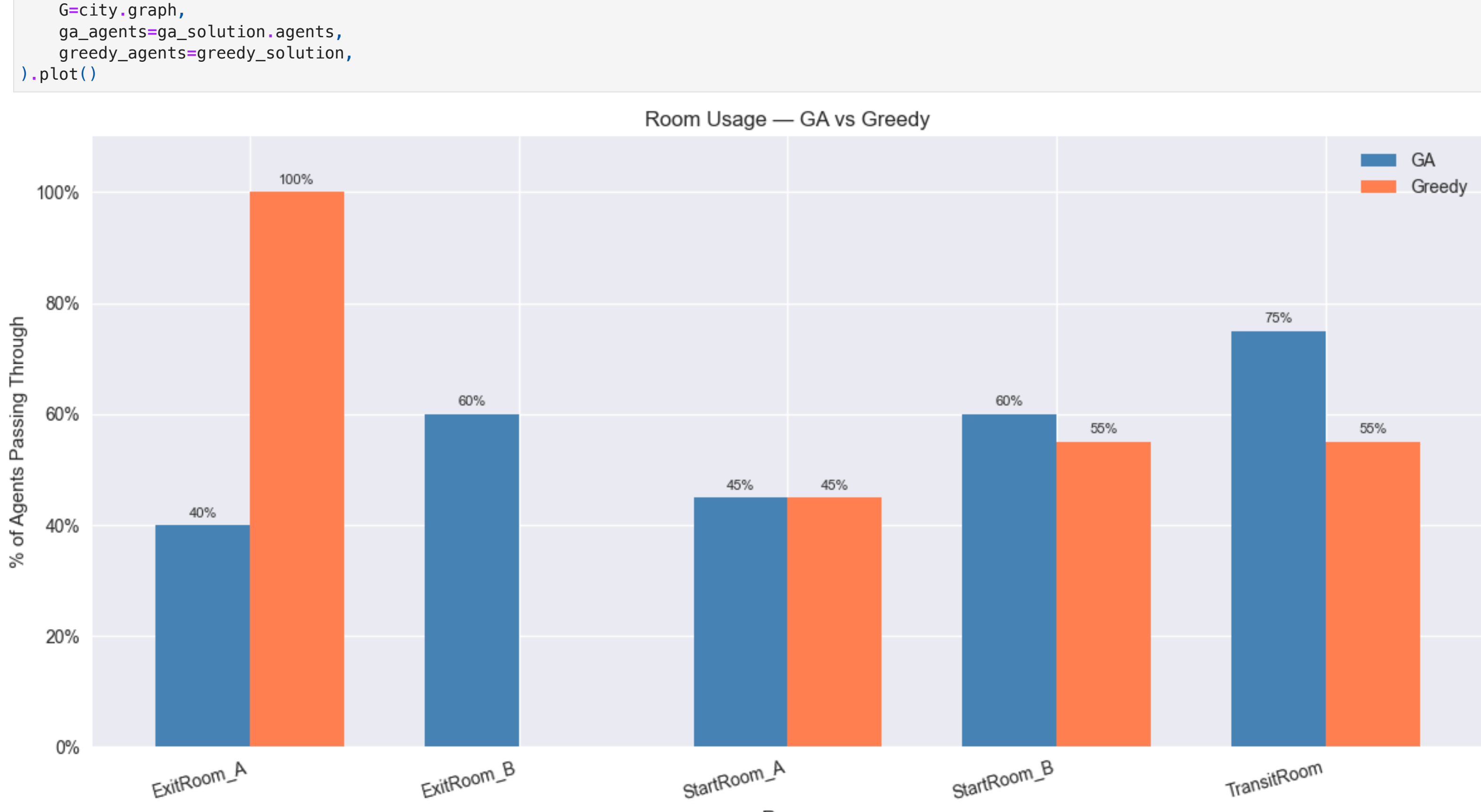
City: bottleneck_severe_small | Population size: 50 | Walking Speed: 1.0

Evolution	1	best fit: 14.000	avg fit: 14.920	worst fit: 16.000	avg path: 7.539	congestion: 1.390	exit use: 0.052	diveristy: 0.044
Evolution	5	best fit: 13.000	avg fit: 14.380	worst fit: 16.000	avg path: 7.560	congestion: 1.354	exit use: 0.049	diveristy: 0.210
Evolution	10	best fit: 11.000	avg fit: 13.820	worst fit: 16.000	avg path: 7.413	congestion: 1.308	exit use: 0.062	diveristy: 0.366
Evolution	15	best fit: 11.000	avg fit: 12.900	worst fit: 15.000	avg path: 7.084	congestion: 1.330	exit use: 0.066	diveristy: 0.409
Evolution	20	best fit: 11.000	avg fit: 12.320	worst fit: 14.000	avg path: 7.284	congestion: 1.066	exit use: 0.056	diveristy: 0.387
Evolution	25	best fit: 11.000	avg fit: 11.900	worst fit: 15.000	avg path: 7.161	congestion: 1.138	exit use: 0.087	diveristy: 0.435
Evolution	30	best fit: 11.000	avg fit: 11.660	worst fit: 13.000	avg path: 7.030	congestion: 1.108	exit use: 0.091	diveristy: 0.474
Evolution	35	best fit: 10.000	avg fit: 12.220	worst fit: 16.000	avg path: 6.870	congestion: 1.180	exit use: 0.087	diveristy: 0.446
Evolution	40	best fit: 10.000	avg fit: 12.340	worst fit: 16.000	avg path: 6.813	congestion: 1.410	exit use: 0.078	diveristy: 0.453
Evolution	45	best fit: 10.000	avg fit: 11.900	worst fit: 15.000	avg path: 6.805	congestion: 1.320	exit use: 0.072	diveristy: 0.446

===== FINAL RESULTS =====
Avg path length: 6.30 | Total time: 10.00 | Average time: 7.30 | Congestion index: 45% | Avg Congestion delay: 1.00 | Exit utilisation: 0.10 | Avg Path Efficiency: 1.28
=====

```
In [7]: from utils.plots import PlotRoomUsage
```

```
PlotRoomUsage(
    G=city.graph,
    ga_agents=ga_solution.agents,
    greedy_agents=greedy_solution,
).plot()
```



Simulations

The below code is to compare the outputs of both algorithms over multiple scenarios. TODO: at the mo this is only for one city need to come back and try different ones.

```
In [8]: greedy_outputs = []
ga_outputs = []
for seed_idx in tqdm(range(n_experiments)):

    _, greedy_evaluation = simulate_greedy_evacuation(
        num_agents=num_agents,
        simulation_params=simulation_params,
        algorithm_seed=algorithm_seeds[seed_idx],
        city=city,
        verbose=False
    )
    greedy_outputs.append(greedy_evaluation)

    _, ga_evaluation = simulate_ga_evacuation(
        city=city,
        num_agents=num_agents,
        population_size=50,
        algorithm_seed=algorithm_seeds[seed_idx],
        max_evolution=50,
        simulation_params=simulation_params,
        verbose=0,
    )
    ga_outputs.append(ga_evaluation)
```

100%|██████████| 30/30 [15:12<00:00, 30.41s/it]

We will compare the following metrics:

- Total evacuation time — is there a statistically significant difference between the time the last agent exits under GA vs greedy? A significant result here is the main finding - the GA either gets everyone out faster or slower overall.
- Average evacuation time — is there a significant difference in typical agent experience? This is more robust than total time because it's less sensitive to a single slow agent skewing the result. If total time differs but average doesn't, one or two agents are driving the difference rather than the whole population.
- Average congestion delay — is the GA significantly reducing the delays agents experience due to overcrowding? This directly tests whether global route optimisation is actually redistributing agents away from bottlenecks. A significant reduction here is the mechanistic explanation for why GA beats greedy — not just that it's faster, but specifically because it reduces congestion.
- Exit utilisation — is there a significant difference in how evenly agents are distributed across exits? This is your core behavioural claim — that greedy causes everyone to crowd the nearest exit while GA spreads agents more evenly. A significant result here confirms the fundamental difference in routing behaviour between the two approaches.
- Average path efficiency — are GA agents taking significantly longer routes than greedy agents? A significant result here combined with better evacuation time tells your central story — GA agents travel less efficiently individually but produce better collective outcomes, which is the price of coordination and the answer to your research question.

```
In [9]: experiment_setup = f'For {city_name.replace("_", " ").capitalize()} with {num_agents} agents across {n_experiments} experiments'
print(experiment_setup)
print(f'={len(experiment_setup)}')
```

For Bottleneck severe small with 20 agents across 30 experiments

```
In [36]: comparison = AlgorithmComparison(greedy_outputs=greedy_outputs, ga_outputs=ga_outputs)
results = comparison.statistical_tests(verbose=True)
```

Total time p-value: 0.000 (Significant: True)
Average time p-value: 0.000 (Significant: True)
Avg Congestion delay p-value: 0.000 (Significant: True)
Exit utilisation p-value: 0.000 (Significant: True)
Avg Path Efficiency p-value: 0.000 (Significant: True)

```
In [ ]: comparison.ga_convergence_plots()
```

The Price of Anarchy (PoA) quantifies the cost of selfish behaviour - specifically, how much worse evacuation performance is when every agent acts greedily compared to globally optimised routing. A PoA of 1.5 means greedy routing takes 50% longer than the GA solution.

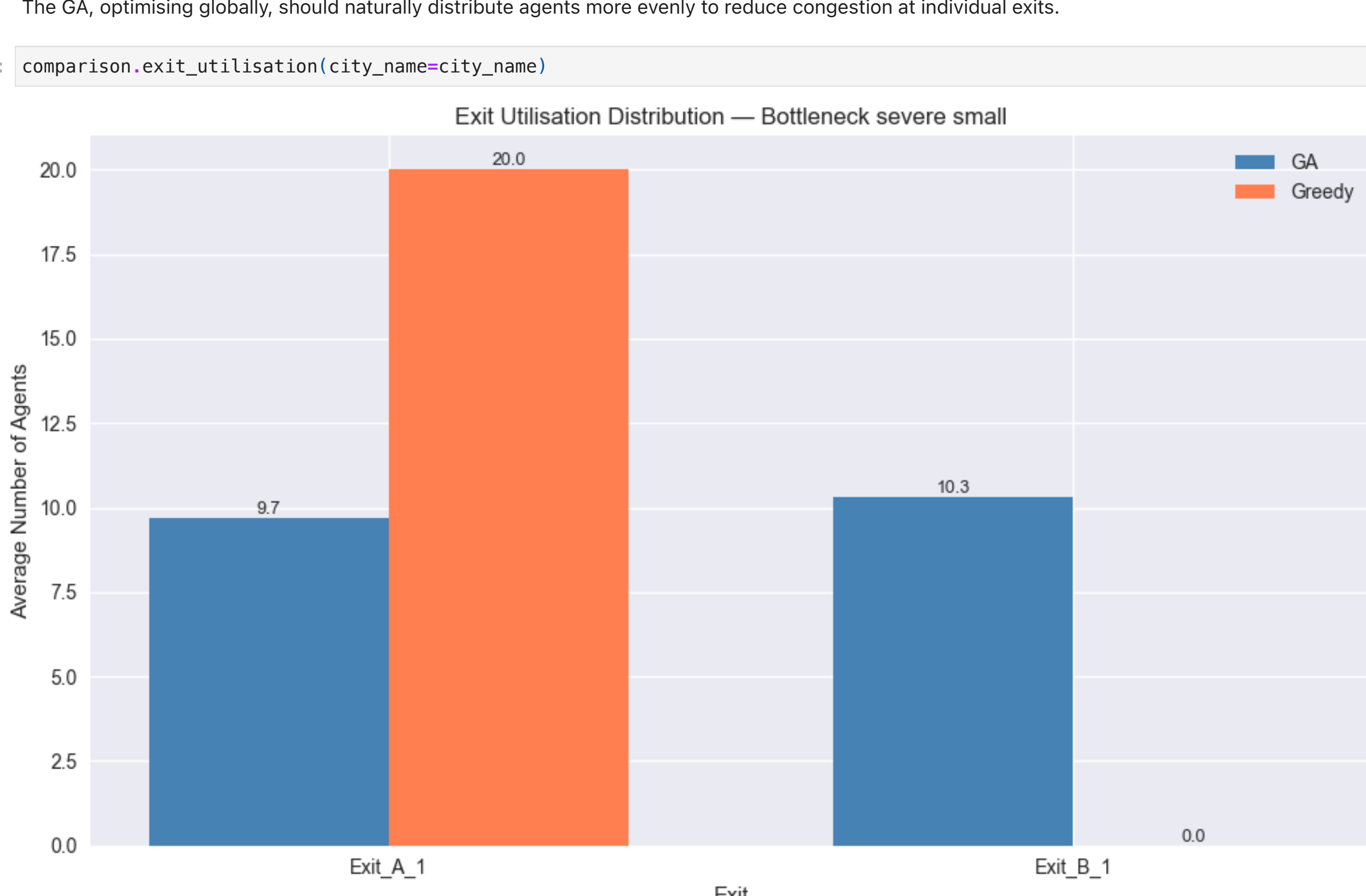
By comparing PoA against each city's betweenness centrality, we test whether the cost of selfish behaviour scales with network bottleneck severity. The hypothesis is that cities with high betweenness centrality (few critical junctions that all shortest paths pass through) will show higher PoA.

```
In [37]: price_of_anarchy = comparison.price_of_anarchy(verbose=True)
```

Price of Anarchy: 3.758 ± 0.655

This compares how agents are distributed across available exits under each approach. Greedy routing is expected to cluster agents at the nearest or most obvious exits, producing uneven utilisation. The GA, optimising globally, should naturally distribute agents more evenly to reduce congestion at individual exits.

```
In [38]: comparison.exit_utilisation(city_name=city_name)
```



This breaks down evacuation performance by where agents begin, rather than reporting a single population average. Some start locations are closer to exits or better connected, so average times will naturally vary.

```
In [39]: comparison.plot_exit_time_by_start_location()
```



```
In [40]: city.start_node_metrics()
```

```
Out[40]: {'Start_A_1': np.float64(3.5),
'Start_A_2': np.float64(4.0),
'Start_B_1': np.float64(4.0),
'Start_B_2': np.float64(4.0)}
```